

Edge-Engine: Um Motor Orquestrador de Processos Concorrentes e Pipelines de Dados para Sistemas POSIX

Daniel Marchioro¹

¹ Departamento de Ciência da Computação
Universidade Federal de Roraima (UFRR)
Boa Vista – RR – Brasil

m4chiorouniv@gmail.com

Abstract. *The growth of Edge Computing and Industrial IoT (IIoT) demands lightweight, resilient orchestration engines able to manage concurrent processes and data pipelines on resource-constrained devices. This paper presents **edge-engine**, a command interpreter and process orchestrator written in C for POSIX systems. The engine provides dual-mode operation (interactive and batch), process isolation via `fork()/execvp()`, UNIX pipelines built over a linear pipe matrix with rigorous descriptor hygiene (`dup2()`, `FD_CLOEXEC` and `close_range()`), I/O redirection (`<`, `>`, `»`), wildcard expansion through `glob()`, a background job table with asynchronous reaping via a race-safe `SIGCHLD` handler, and a producer/consumer message-injection syntax (`<=`, `=>`) implemented over **POSIX message queues**. Validation across nine batch test suites confirmed correct process lifecycle management, zero zombie proliferation and full memory/descriptor hygiene (`valgrind`: 0 leaks, 0 errors).*

Resumo. *O crescimento da Computação de Borda (Edge Computing) e da Internet das Coisas Industrial (IIoT) exige motores de orquestração leves e resilientes, capazes de gerenciar processos concorrentes e pipelines de dados em dispositivos com recursos limitados. Este artigo apresenta o **edge-engine**, um interpretador de comandos e orquestrador de processos desenvolvido em C para sistemas POSIX. O motor oferece operação em modo duplo (interativo e em lote), isolamento de processos via `fork()/execvp()`, pipelines UNIX construídos sobre uma matriz linear de pipes com higiene rigorosa de descritores (`dup2()`, `FD_CLOEXEC` e `close_range()`), redirecionamento de E/S (`<`, `>`, `»`), expansão de metacaracteres via `glob()`, uma tabela de jobs em background com coleta assíncrona por um tratador de `SIGCHLD` protegido contra condições de corrida, e uma sintaxe de injeção produtor/consumidor (`<=`, `=>`) implementada sobre **filas de mensagens POSIX**. A validação por nove suítes de teste em lote confirmou o gerenciamento correto do ciclo de vida dos processos, ausência de proliferação de zumbis e higiene completa de memória e descritores (`valgrind`: 0 vazamentos, 0 erros).*

1. Introdução

A consolidação da Computação de Borda (*Edge Computing*) e da Internet das Coisas Industrial (IIoT) impõe requisitos severos sobre o software de infraestrutura: processamento local de fluxos de dados de sensores, consumo mínimo de recursos e resiliência

a falhas, tudo em dispositivos com memória e capacidade de processamento limitadas [Silberschatz et al. 2018]. Nesse contexto, *frameworks* pesados e *runtimes* gerenciados são inviáveis; a alternativa é desenvolver soluções nativas e modulares que explorem diretamente as primitivas do sistema operacional: chamadas de sistema (*system calls*), descritores de arquivos, mecanismos de comunicação interprocessos (IPC) e sinais POSIX.

O desafio central consiste em criar um mecanismo capaz de ler planos de execução, disparar concorrentemente as tarefas necessárias, encadear o fluxo de dados em memória e garantir que nenhum recurso do sistema operacional seja desperdiçado ou alocado indefinidamente. A ausência de coleta dos estados de término de processos filhos, por exemplo, origina *processos zumbis* que consomem entradas na tabela de processos do *kernel* até o esgotamento do recurso.

Este trabalho apresenta o **edge-engine**, um motor orquestrador de processos concorrentes e pipelines de dados implementado em C para sistemas POSIX (Linux). A ferramenta opera em dois modos complementares — interativo e autônomo em lote — e oferece, além das primitivas clássicas de *shell* (pipelines, redirecionamentos, *wildcards*, *jobs* em background e comandos internos), uma sintaxe de injeção por mensagens produtor/consumidor ($\leq/\geq$) construída sobre **filas de mensagens POSIX**, um mecanismo de IPC por troca de mensagens distinto do `pipe()` e adequado ao desacoplamento temporal típico de aplicações de telemetria de borda.

Este relatório está organizado conforme a documentação de Engenharia de Sistemas Operacionais exigida para o projeto. As Seções 3 a 9 cobrem rigorosamente os sete tópicos técnicos: (1) análise arquitetural do terminal; (2) ciclo de vida de processos; (3) engenharia de concorrência e sinais; (4) geometria de pipes e redirecionamento; (5) expansão de metacaracteres; (6) codificação de caracteres e internacionalização; e (7) portfólio de comandos internos. As Seções seguintes tratam da implementação, dos resultados experimentais e das conclusões.

O repositório do projeto está disponível em: https://github.com/Marchioro-dM/FP_DCC403_2_2026.1_UFRR

2. Fundamentação Teórica

2.1. Modelo de Processos POSIX

No modelo POSIX, cada processo possui um espaço de endereçamento isolado, um conjunto de descritores de arquivos abertos e um estado de execução. A criação de processos segue o paradigma *fork-exec*: a chamada `fork()` clona o processo pai, criando um filho com cópia lógica do espaço de endereçamento (com *copy-on-write*); a família `exec()` substitui a imagem do processo filho por um novo programa, preservando os descritores herdados que não estejam marcados com `FD_CLOEXEC`.

Formalmente, seja P o conjunto de processos ativos. Após `fork()`:

$$P' = P \cup \{p_{\text{filho}}\}, \quad \text{onde} \quad \text{env}(p_{\text{filho}}) = \text{env}(p_{\text{pai}}) \quad (1)$$

2.2. Comunicação Interprocessos: Pipes e Filas de Mensagens

Um **pipe anônimo** é um canal unidirecional criado por `pipe(int fd[2])`, retornando `fd[0]` (leitura) e `fd[1]` (escrita). É o mecanismo usado pelo operador `|`: para um

pipeline de n comandos $C_1 \mid \dots \mid C_n$ são necessários $n - 1$ pipes $\{p_1, \dots, p_{n-1}\}$, com:

$$\text{stdout}(C_k) \rightarrow p_k[1] \quad \text{e} \quad p_k[0] \rightarrow \text{stdin}(C_{k+1}), \quad 1 \leq k < n. \quad (2)$$

Já uma **fila de mensagens POSIX** (`mq_open`, `mq_send`, `mq_receive`) é um objeto IPC *nomeado* e persistente no *kernel*, que preserva fronteiras de mensagem e permite desacoplamento temporal entre produtor e consumidor. O edge-engine emprega esse segundo mecanismo — e não pipes nomeados (`mkfifo`) — para implementar os operadores de injeção $\leq$ e $\geq$ (Seção 6).

2.3. Processos Zumbis e o Sinal SIGCHLD

Quando um processo filho termina, ele permanece no estado *zumbi* (Z) até que o pai leia seu código de saída via `wait()` ou `waitpid()`. Para processos em *foreground*, o pai bloqueia sincronamente. Para processos em *background*, o tratamento é assíncrono: o *kernel* envia SIGCHLD ao pai a cada término de filho. O tratador deve invocar `waitpid(-1, &status, WNOHANG)` em laço até esgotar os filhos terminados:

$$\forall p_i \in P_{\text{bg}} : \text{terminated}(p_i) \Rightarrow \text{waitpid}(p_i) \text{ é chamado.} \quad (3)$$

3. Tópico 1 — Análise Arquitetural do Terminal

Um interpretador de comandos moderno separa responsabilidades em dois subsistemas distintos: o **Parser**, responsável pela análise léxico-sintática da linha de entrada, e o **Executor**, responsável pela orquestração dos processos. O edge-engine adota essa separação e a materializa em módulos independentes, ligados por um cabeçalho global (`include/edge_engine.h`) que define as estruturas de dados compartilhadas. O fluxo macro de execução é apresentado na Figura 1.

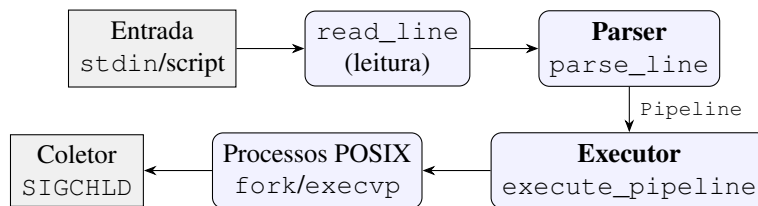


Figura 1. Visão macro (*Big Picture*) da arquitetura do edge-engine: o Parser converte a linha em uma estrutura `Pipeline`, consumida pelo Executor, que dispara os processos POSIX cujos terminos são coletados de forma assíncrona pelo tratador de SIGCHLD.

3.1. Responsabilidades do Parser

O Parser (`src/parser/`) processa a linha caractere a caractere e produz uma estrutura `Pipeline`. A análise ocorre em camadas (`parse_line`): primeiro destaca-se o sufixo de background (`&`); em seguida `detect_operators` extrai os operadores de fila (`<=`/`=>`); `split_pipeline` segmenta os estágios pelo caractere `|`; e, por fim, `parse_command` tokeniza cada estágio, tratando aspas (`"`/`'`) e os redirecionamentos (`<`, `>`, `»`). As estruturas centrais são:

```

1 typedef struct {
2     char *args[MAX_ARGS]; /* argumentos (NULL-terminado) */
3     int  argc;             /* contagem de argumentos */
4     char *input_file;      /* redirecionamento < */
5     char *output_file;     /* redirecionamento > ou >> */
6     int  append;           /* 1 se >> (append) */
7     int  background;       /* execucao em background (&) */
8 } Command;
9
10 typedef struct {
11     Command commands[MAX_PIPES];
12     int  num_commands;
13     int  background;
14     char *producer;      /* fila do operador <= */
15     char *consumer;      /* fila do operador => */
16     int  has_producer, has_consumer;
17     char *raw;           /* linha original (rotulo job) */
18 } Pipeline;

```

Listing 1. Estruturas de representação (include/edge_engine.h)

3.2. Responsabilidades do Executor

O Executor (src/executor/) recebe o Pipeline e orquestra a execução: cria a matriz de pipes, decide entre comando interno e externo, realiza os *forks*, configura redirecionamentos e descritores, e gerencia a espera (*foreground*) ou o registro de *job* (*background*). O Executor **não** interpreta sintaxe — essa responsabilidade é exclusiva do Parser —, o que mantém os módulos desacoplados e auditáveis. Módulos auxiliares completam a arquitetura: src/builtins/ (comandos internos), src/utils/ (*wrappers* seguros *safe_close*, *safe_dup2*, *strdup_safe*) e src/main.c (loop principal, sinais e a tabela de *jobs*).

4. Tópico 2 — Mecanismo de Ciclo de Vida de Processos

A execução de cada comando externo segue o paradigma *fork-exec*. A função *execute_command* primeiro verifica se o comando é interno e pode ser executado diretamente no pai (sem pipes nem redirecionamentos); caso contrário, realiza o *fork()*. **O momento exato do isolamento de memória** é o retorno de *fork()*: a partir dele, pai e filho passam a operar sobre espaços de endereçamento logicamente independentes (fisicamente compartilhados via *copy-on-write* até a primeira escrita). No ramo do filho (*pid == 0*), a imagem é substituída por *execvp()*, que **só retorna em caso de erro**; a partir desse ponto o código do edge-engine deixa de existir no processo.

```

1 fflush(NULL);
2 pid_t pid = fork(); /* <-- isolamento de memoria */
3 if (pid == 0) {
4     /* Filho em foreground deve poder ser interrompido */
5     signal(SIGINT, SIG_DFL);
6     signal(SIGQUIT, SIG_DFL);
7
8     /* Redirecionamentos de entrada/saida */
9     if (cmd->input_file) { int fd = open(cmd->input_file, O_RDONLY);
10                          dup2(fd, STDIN_FILENO); }

```

```

11     else if (in_fd >= 0) dup2(in_fd, STDIN_FILENO);
12     if (cmd->output_file) { int flags = O_WRONLY|O_CREAT|
13                             (cmd->append?O_APPEND:O_TRUNC);
14                             int fd = open(cmd->output_file, flags,
15                                         0644);
16                             dup2(fd, STDOUT_FILENO); }
17     else if (out_fd >= 0) dup2(out_fd, STDOUT_FILENO);
18
19     /* Fecha todos os fds herdados acima de stderr de uma vez */
20     close_range(3, ~0U, 0);
21
22     expand_wildcards(cmd); /* expansao glob (Topico 5) */
23     execvp(cmd->args[0], cmd->args); /* substitui a imagem */
24
25     /* execvp so retorna em erro: mapeia errno -> codigo de saida */
26     if (errno == ENOENT) exit(127); /* comando nao encontrado */
27     if (errno == EACCES) exit(126); /* permissao negada */
28     exit(EXIT_FAILURE);
29 }

```

Listing 2. Ciclo de vida no filho: isolamento, configuração e substituição de imagem (executor.c, resumido)

O mapeamento de `errno` para os códigos de saída convencionais (127 para comando inexistente, 126 para permissão negada) preserva a semântica esperada de um *shell* POSIX. No pai, o *pid* retornado é registrado para posterior espera ou para a tabela de *jobs*, conforme o modo de execução (Seção 5).

5. Tópico 3 — Engenharia de Concorrência e Sinais

5.1. Tratador Assíncrono de SIGCHLD

O tratador `handle_sigchld` implementa a coleta não-bloqueante de todos os filhos terminados, conforme a Equação 3. Por executar em contexto de sinal, ele usa apenas operações *async-signal-safe*: `waitpid` e escritas em inteiros, além de salvar e restaurar `errno` para não corromper o fluxo interrompido. Cada *pid* coletado decrementa o contador de processos pendentes do *job* correspondente na tabela.

```

1 void handle_sigchld(int sig) {
2     (void) sig;
3     int saved_errno = errno;
4     pid_t pid; int status;
5     while ((pid = waitpid(-1, &status, WNOHANG)) > 0) {
6         if (pid == foreground_pid) foreground_pid = -1;
7         child_status = status;
8         for (int j = 0; j < MAX_JOBS; j++) { /* atualiza job */
9             if (!job_table[j].used) continue;
10            for (int k = 0; k < job_table[j].npids; k++)
11                if (job_table[j].pids[k] == pid) {
12                    if (job_table[j].nremaining > 0) job_table[j].
13                        nremaining--;
14                    job_table[j].last_status = status;
15                }
16        }
17    }
18 }

```

```

17     errno = saved_errno;
18 }
19
20 void setup_signal_handlers(void) {
21     struct sigaction sa;
22     sigemptyset(&sa.sa_mask);
23     sa.sa_flags = SA_RESTART | SA_NOCLDSTOP;
24     sa.sa_handler = handle_sigchld;
25     sigaction(SIGCHLD, &sa, NULL);
26     signal(SIGINT, SIG_IGN); /* motor nao morre com Ctrl-C */
27     signal(SIGQUIT, SIG_IGN); /* motor nao morre com Ctrl-\\ */
28 }

```

Listing 3. Tratador de SIGCHLD e seu registro (main.c, resumido)

A flag `SA_NOCLDSTOP` evita a entrega de `SIGCHLD` por *paradas* (apenas terminos interessam) e `SA_RESTART` reinicia chamadas de sistema lentas interrompidas. O pai ignora `SIGINT` e `SIGQUIT` para sobreviver às combinações de teclado, enquanto cada filho em *foreground* restaura o comportamento padrão (Seção 4), podendo ser interrompido normalmente.

5.2. Prevenção de Condições de Corrida

Um ponto delicado é a interação entre o laço principal e o tratador assíncrono sobre a tabela de *jobs* e sobre a espera por *foreground*. O edge-engine bloqueia `SIGCHLD` (via `sigprocmask`) ao redor de duas regiões críticas: (i) entre o `fork` e o `waitpid` dos processos em *foreground* — evitando que o tratador recolha o filho antes do `waitpid` específico, o que causaria `ECHILD` e perda do código de saída; e (ii) entre o `fork` e o registro do *job* em *background* — evitando que um filho seja coletado antes de o *job* existir na tabela, deixando-o eternamente “em execução”. As funções `add_job`, `report_done_jobs` e `list_jobs` também bloqueiam o sinal ao manipular a tabela.

5.3. Diagrama de Estados: Como o waitpid Evita Zumbis

A Figura 2 formaliza o ciclo de vida do processo filho do ponto de vista do pai. O estado *Zumbi* é transitório: ele só persiste se o pai nunca chamar `wait/waitpid`. No edge-engine, ambos os caminhos — o `waitpid` síncrono do *foreground* e o `waitpid(-1, WNOHANG)` do tratador de `SIGCHLD` — garantem a coleta, eliminando a transição para o estado terminal de *Zumbi Persistente*.

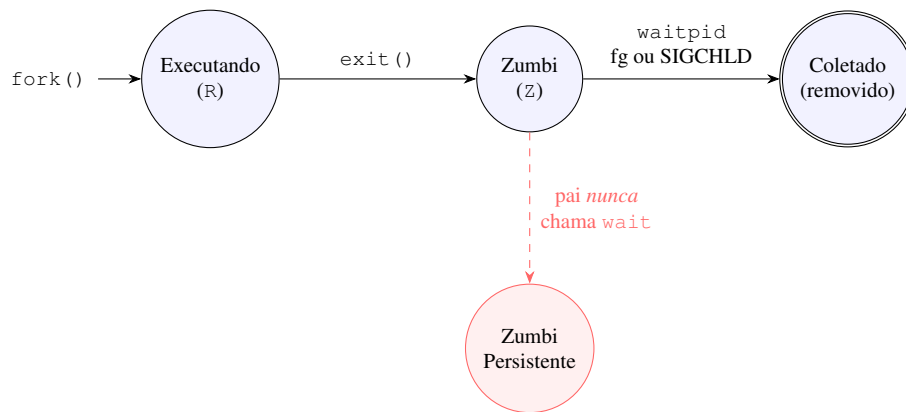


Figura 2. Diagrama de estados do ciclo de vida de um filho sob a ótica do pai. A coleta por `waitpid` (síncrona em *foreground* ou assíncrona via `SIGCHLD` em *background*) leva ao estado *Coletado*; a ausência de coleta (tracejado vermelho) levaria ao *Zumbi Persistente*, situação que o edge-engine previne por construção.

6. Tópico 4 — Geometria de Pipes e Redirecionamento

6.1. Matriz Linear de Pipes

Para um pipeline de n comandos, o Executor aloca uma matriz `pipe_fds[MAX_PIPES][2]` com $n - 1$ pares de descritores (Equação 2). Cada par recebe `FD_CLOEXEC` via `fcntl`, de modo que os descritores são fechados automaticamente em qualquer `execvp` bem-sucedido — uma defesa adicional contra o vazamento de descritores:

```

1 int setup_pipes(int pipe_fds[][2], int num_pipes) {
2     for (int i = 0; i < num_pipes; i++) {
3         if (pipe(pipe_fds[i]) == -1) { perror("pipe"); return -1; }
4         fcntl(pipe_fds[i][0], F_SETFD, FD_CLOEXEC);
5         fcntl(pipe_fds[i][1], F_SETFD, FD_CLOEXEC);
6     }
7     return 0;
8 }

```

Listing 4. Criação da matriz de pipes com `FD_CLOEXEC` (executor.c)

6.2. Mapeamento de Descritores via `dup2` e `close_range`

Para o estágio C_i , o Executor seleciona a extremidade de leitura do pipe anterior (`in_fd`) e a de escrita do pipe seguinte (`out_fd`); no filho, `dup2` reposiciona esses descritores sobre `STDIN/STDOUT`. Em seguida, uma única chamada `close_range(3, ~0U, 0)` fecha de uma só vez todos os descritores herdados acima de `stderr`, cobrindo todos os pipes da matriz sem risco de *double-close*. A Figura 3 ilustra o mapeamento para um pipeline de três estágios.

6.3. Redirecionamento de Arquivos e Pipes Dedicados

O redirecionamento de E/S é resolvido no mesmo ponto: `<` abre o arquivo em `O_RDONLY` e o duplica sobre `STDIN`; `>` e `>>` abrem em `O_WRONLY|O_CREAT` com `O_TRUNC`

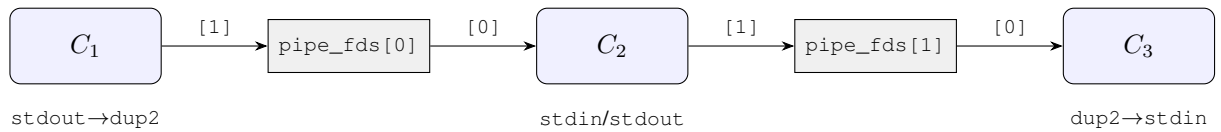


Figura 3. Geometria de um pipeline de três estágios. A escrita `pipe_fds[k][1]` recebe o `stdout` de C_k e a leitura `pipe_fds[k][0]` alimenta o `stdin` de C_{k+1} , conforme a Equação 2.

ou `O_APPEND`, respectivamente, duplicando sobre `STDOUT`. Quando o pipeline usa os operadores de fila (`<=>`), o Executor cria *pipes dedicados* (`producer_pipe`, `consumer_pipe`) que ligam o primeiro e o último estágio aos processos de drenagem/publicação das filas POSIX descritos na Seção 9.

7. Tópico 5 — Expansão de Metacaracteres (Wildcards)

A expansão de padrões de arquivos é realizada pela função `expand_wildcards`, executada no processo filho imediatamente antes do `execvp`. Cada argumento é inspecionado por `strpbrk` em busca dos metacaracteres `*`, `?` ou `[]`; havendo correspondência, a API nativa POSIX `glob()` expande o padrão. A *flag* `GLOB_NOCHECK` preserva o comportamento de *shell* (um padrão sem correspondência é mantido literalmente), e `GLOB_TILDE` expande `~`:

```

1 int expand_wildcards(Command *cmd) {
2     int new_argc = 0;
3     char *new_args[MAX_ARGS] = {0};
4     for (int i = 0; i < cmd->argc; i++) {
5         if (strpbrk(cmd->args[i], "*?[]") != NULL) {
6             glob_t g;
7             if (glob(cmd->args[i], GLOB_NOCHECK | GLOB_TILDE, NULL, &g)
8                 == 0) {
9                 for (size_t j = 0; j < g.gl_pathc && new_argc <
10                     MAX_ARGS-1; j++)
11                     new_args[new_argc++] = strdup(g.gl_pathv[j]);
12                 globfree(&g);
13             } else {
14                 new_args[new_argc++] = strdup(cmd->args[i]);
15             }
16         } else {
17             new_args[new_argc++] = strdup(cmd->args[i]);
18         }
19     }
20     /* substitui args antigos pelos expandidos, mantendo NULL-
21        terminacao */
22     for (int i = 0; i < cmd->argc; i++) { free(cmd->args[i]); cmd->args
23         [i]=NULL; }
24     for (int i = 0; i < new_argc; i++) cmd->args[i] = new_args[i];
25     cmd->argc = new_argc; cmd->args[new_argc] = NULL;
26     return 0;
27 }

```

Listing 5. Expansão de wildcards via `glob()` (executor.c)

Padrões como `*.dat`, `edge_wc_sensor*` e `edge_wc_sensor?.dat` foram validados (Seção 11). Por expandir no filho e logo antes do `exec`, a expansão não

polui o estado do processo pai e respeita o limite estático `MAX_ARGS`.

8. Tópico 6 — Codificação de Caracteres e Internacionalização

8.1. Tokenização Orientada a Bytes

O Parser trata a entrada como sequência de bytes, usando `isspace((unsigned char)c)` para delimitar tokens e `strpbrk/strcmp` para reconhecer operadores. Como os delimitadores e operadores do edge-engine (espaço, `|`, `&`, `<`, `>`, `=`) são todos caracteres ASCII (*code points* $< 0x80$), e como o UTF-8 é *self-synchronizing* — nenhum byte de continuação de uma sequência multibyte cai na faixa ASCII —, a segmentação por esses delimitadores é **segura por byte** mesmo para nomes de arquivos em UTF-8: um nome como `relatório.txt` é mantido íntegro como um único token. O conteúdo trafegado pelos pipes e filas é tratado como binário opaco, não sofrendo interpretação de *charset*.

8.2. Buffers Estáticos e Prevenção de Buffer Overflow

O motor adota buffers de tamanho fixo (`MAX_LINE_LENGTH = 4096`, `MAX_ARGS = 64`, `MAX_PIPES = 32`), uma escolha deliberada para dispositivos de borda: previsibilidade de memória e ausência de fragmentação de *heap*. A integridade é garantida por leitura limitada e cópias controladas: `read_line` usa `fgets(buffer, sizeof(buffer), ...)` (nunca ultrapassa o buffer), a tokenização verifica `argc < MAX_ARGS - 1` antes de cada inserção, e as cópias usam `strncpy` com terminação explícita. Assim, entradas maiores que os limites são *truncadas com segurança*, sem corrupção de memória nem *buffer overflow*.

8.3. Detalhe de Buffering do stdin em Modo Batch

Um cuidado de internacionalização de fluxo (e não apenas de *charset*) é o *buffering* do *stream* de entrada. Em modo *batch*, com a entrada vinda de arquivo, o `fgets` com *buffering* padrão consumiria um bloco inteiro do `stdin`, “roubando” bytes que um filho (p.ex. `grep` lendo `stdin`) deveria receber. O edge-engine resolve isso com `setvbuf(input, NULL, _IONBF, 0)`, lendo apenas até o `'\n'` e preservando o restante no descritor para o filho. A migração dos buffers estáticos para alocação dinâmica permanece como trabalho futuro (Seção 12).

9. Tópico 7 — Portfólio de Comandos Suportados

9.1. Comandos Internos (Built-ins)

Certos comandos precisam alterar o estado do próprio processo orquestrador e, por isso, são executados *no pai* (não podem ser delegados a um filho): mudar de diretório afeta o *cwd* do motor, encerrar deve terminar o próprio motor, e listar *jobs* requer acesso à tabela mantida no processo pai. O despacho é feito por `is_builtin/execute_builtin`. A Tabela 1 lista o portfólio completo de seis comandos internos.

9.2. Built-ins em Pipelines

Quando um comando interno participa de um pipeline ou usa redirecionamento, ele é executado em um filho dedicado, de modo que os redirecionamentos de descritores funcionem sem afetar o processo pai. Fora desse caso (comando isolado, sem pipes nem redirecionamentos), o *built-in* roda diretamente no pai, preservando efeitos colaterais como a mudança de diretório de `cd`.

Tabela 1. Portfólio de comandos internos do edge-engine

Comando	Assinatura	Descrição / Escopo
cd	cd [dir]	Muda o diretório de trabalho via <code>chdir</code> ; sem argumento, vai para <code>\$HOME</code> .
pwd	pwd	Imprime o diretório atual (<code>getcwd</code>).
echo	echo ...	Escreve os argumentos separados por espaço, seguidos de nova linha.
exit	exit	Encerra o motor (<code>exit(0)</code>).
fim	fim	Sinônimo de <code>exit</code> ; encerra o motor.
jobs	jobs	Lista os <i>jobs</i> em background ainda em execução (<code>list_jobs</code>).

9.3. Operadores de Injeção Produtor/Consumidor sobre Filas POSIX

Além dos *built-ins*, o núcleo do orquestrador trata diretamente os operadores de injeção por mensagens, implementados sobre **filas de mensagens POSIX** (`mq_open/mq_send/mq_receive`). O operador `=>` (consumidor) publica o `stdout` do pipeline como mensagens em uma fila nomeada; o operador `<=` (produtor) drena a fila nomeada de forma não-bloqueante e injeta seu conteúdo no `stdin` do pipeline, removendo a fila (`mq_unlink`) ao final para não desperdiçar recursos do *kernel*. A forma `<= origem => destino` estabelece uma ponte direta fila→fila. A persistência das filas no *kernel* permite desacoplamento temporal entre produtor e consumidor, padrão recorrente em telemetria de borda:

```
echo "ERROR: sensor" => log_sensor # publica na fila /log_sensor
grep ERROR <= log_sensor           # drena a fila e filtra
cat <= origem => destino           # ponte direta fila -> fila
```

Listing 6. Operadores de fila POSIX

10. Implementação

O *edge-engine* foi implementado em C com extensões GNU (`#define _GNU_SOURCE`; uso de `close_range` e `GLOB_TILDE`) e compilado com `gcc` sob `-Wall -Wextra -Werror -g -fPIE`, ligando contra `-lm -lrt` (a biblioteca `rt` fornece as filas de mensagens POSIX). A ausência de dependências além da *libc* POSIX favorece a portabilidade para dispositivos embarcados. O projeto organiza-se em:

- `include/edge_engine.h` — cabeçalho global (estruturas, constantes);
- `src/main.c` — loop principal, sinais, leitura de linha e tabela de *jobs*;
- `src/parser/` — Tópico 1: análise léxico-sintática;
- `src/executor/` — Tópicos 2, 4 e 5: ciclo de vida, pipes, redirecionamento, *wildcards* e filas POSIX;
- `src/builtins/` — Tópico 7: comandos internos;
- `src/utils/` — *wrappers* seguros (`safe_close`, `safe_dup2`, `strdup_safe`);
- `Makefile` — automação de compilação e do alvo `valgrind`.

O desenvolvimento, a refatoração e a depuração do código-fonte, bem como a geração dos *scripts* de teste em lote, contaram com o suporte assistido do modelo de linguagem Claude Opus 4.8 (Anthropic); toda a lógica algorítmica, a parametrização e a validação foram auditadas manualmente pelo autor, conforme detalhado na Seção 14.

```
make                # compila -> bin/edge-engine
make valgrind       # roda os scripts de teste sob valgrind
./bin/edge-engine    # modo interativo
./bin/edge-engine script.txt # modo em lote
```

11. Estudo de Casos e Resultados

A validação foi conduzida por nove *scripts* de teste em lote (tests/test_scripts/), cada um exercitando um aspecto do motor. Todos foram executados sob valgrind -leak-check=full -show-leak-kinds=all, com os relatórios arquivados em tests/valgrind_logs/.

11.1. Cenário de Pipeline e Wildcards

```
[edge-orchester] » echo hello edge engine | wc -w
3
[edge-orchester] » ls /tmp/edge_wc_*.dat
/tmp/edge_wc_sensor1.dat  /tmp/edge_wc_sensor2.dat  /tmp/
edge_wc_sensor3.dat
```

Listing 7. Trechos de test_pipes.txt e test_wildcards.txt

11.2. Cenário de Filas POSIX e Background

```
[edge-orchester] » printf "ERROR: temp\nINFO: ok\n" => telemetry_raw
[edge-orchester] » grep ERROR <= telemetry_raw
ERROR: temp
[edge-orchester] » sleep 2 &
[1] 4821
[edge-orchester] » jobs
[1]+  Executando  sleep 2 &
```

Listing 8. Trechos de test_operadores_especiais.txt e test_background.txt

11.3. Resumo dos Resultados

A Tabela 2 consolida os resultados. Em todas as suítes, o valgrind reportou **0 bytes em uso na saída** e **0 erros**, confirmando a higiene completa de memória e descritores e a ausência de processos zumbis.

12. Discussão e Limitações

A implementação cobre os requisitos essenciais do orquestrador de borda, com limitações intencionais que delimitam seu escopo. Os **buffers estáticos** (MAX_LINE_LENGTH, MAX_ARGS, MAX_PIPES) truncam linhas e pipelines muito extensos; a migração para alocação dinâmica é o principal trabalho futuro. O cd realiza chdir sobre o argumento literal, não implementando atalhos como cd - (retorno ao diretório anterior). Não há, no escopo atual, expansão de variáveis de ambiente (p.ex. \$!), operadores condicionais (&&, ||) nem substituição de comandos. A tokenização orientada a bytes preserva nomes UTF-8 nos casos comuns (Seção 8), mas não normaliza *code points* nem trata *collation* dependente de *locale*.

Tabela 2. Resultados da suíte de testes do edge-engine (valgrind: 0 vazamentos, 0 erros em todas)

Suíte de Teste	Aspecto Exercitado	Resultado
test_pipes	Pipelines (2–4 estágios)	✓
test_redirecionamento	<, >, », combinados	✓
test_background	&, tabela de <i>jobs</i> , reporte	✓
test_builtins	cd, pwd, echo	✓
test_wildcards	glob(): *, ?, prefixo	✓
test_operadores_especiais	Filas POSIX <=, =>, ponte	✓
test_telemetria	Ciclo IIoT (pipes + filas + redir.)	✓
test_robustez	Erros, comandos inválidos, resiliência	✓
test_stress	Concorrência e rajadas de comandos	✓

13. Trabalhos Relacionados

O *Bash* e o *dash* são os interpretadores de comandos POSIX mais difundidos, com suporte completo a pipelines, redirecionamentos e controle de *jobs*. O edge-engine difere ao priorizar um subconjunto mínimo e auditável voltado ao contexto de borda, acrescido dos operadores de injeção por filas de mensagens POSIX, ausentes nos *shells* tradicionais. O *BusyBox* agrupa *shell* e utilitários em um binário compacto para sistemas embarcados, mas não oferece o mecanismo produtor/consumidor sobre filas nomeadas. Trabalhos sobre orquestração leve para IIoT, como, motivam a necessidade de soluções nativas sem dependência de *runtimes* pesados.

14. Conclusão

Este trabalho apresentou o **edge-engine**, um motor orquestrador de processos concorrentes e pipelines de dados para sistemas POSIX. A documentação cobriu rigorosamente os sete tópicos exigidos: a separação Parser/Executor; o ciclo de vida *fork/execvp* com isolamento de memória; a engenharia de concorrência com tratamento assíncrono e *race-safe* de SIGCHLD; a geometria de pipes com matriz linear, *dup2* e *close_range*; a expansão de *wildcards* via *glob()*; o tratamento de codificação e buffers com prevenção de *overflow*; e o portfólio de comandos internos. A validação por nove suítes de teste, todas com *valgrind* limpo (0 vazamentos, 0 erros), confirmou a ausência de zumbis e a higiene completa de recursos. Como trabalho futuro, propõem-se a alocação dinâmica de buffers, os operadores condicionais (&&, ||), a expansão de variáveis de ambiente e a descoberta dinâmica de agentes em rede.

Declaração de Uso de Inteligência Artificial

Em conformidade com as diretrizes de transparência aplicáveis, declara-se o uso de ferramenta de Inteligência Artificial generativa na elaboração deste trabalho. **Ferramenta:** Claude Opus 4.8, desenvolvida pela Anthropic. **Escopo do uso:** (i) revisão linguística e aprimoramento estilístico do texto deste relatório; (ii) suporte assistido à escrita, refatoração e correção do código-fonte do edge-engine; e (iii) geração dos *scripts* de teste em lote utilizados na validação. Todo o conteúdo conceitual, a lógica algorítmica, a parametrização, a interpretação dos resultados e a redação final foram rigorosamente supervisionados,

auditados e validados pelo autor humano, que assume total responsabilidade pelo conteúdo apresentado.

Referências

[Silberschatz et al. 2018] Silberschatz, A., Galvin, P. B., and Gagne, G. (2018). *Operating System Concepts*. Wiley, 10th edition.